

Aufwandsschätzung StudyQuest – Finalschätzung

Titel des Dokuments:

Aufwandsschätzung Final - StudyQuest

Projektname:

StudyQuest – Gamifizierte Lern- und Notenverwaltungs-Web-App

Modul:

Software Engineering I – Praxis

Projektzeitraum:

Wintersemester 2025 / 2026

Version:

1.2

Datum:

03.03.2026

Author:innen:

- Michael Steer (Scrum Master)
- Luke Engehardt (Product Owner)
- Giuliana Carrano (Developer)
- Paul Strasser (Developer)
- Roman Faber (Developer)

Betreuer / Prüfer:

Sascha Wanninger

Freigabe durch autorisierte Person:

Michael Steer (Scrum Master)

Changelog

Version	Datum	Autor	Änderungsbeschreibung
0.1	01.02.2026	M. Steer	Datenerhebung: Ist-Aufwände pro Use Case aus Sprint-Logs erfasst
1.0	06.02.2026	M. Steer	Erstfassung der Aufwandsschätzung (Final) mit Abweichungsanalyse

Version	Datum	Autor	Änderungsbeschreibung
1.1	25.02.2026	M. Steer	Diskussion Initial vs. Final erweitert, Hauptgründe für Abweichung dokumentiert
1.2	03.03.2026	M. Steer	Finalversion zur Abgabe vorbereitet

Distribution List

Name	Rolle	Kommentar / Zuständigkeit
Sascha Wanninger	Prüfer / Betreuer	Bewertung im Rahmen des Moduls
Michael Steer	Scrum Master	Koordination & Freigabe
Luke Engehardt	Product Owner	Anforderungen, Dokumentation & Technische Dokumentation
Giuliana Carrano	Developer	Projektskizze, Dokumentation, Architektur & UML
Paul Strasser	Developer	
Roman Faber	Developer	

1. Vergleich: Geschätzt vs. Tatsächlich

Phase	Geschätzt (h)	Tatsächlich (h)	Abweichung (h)	Abweichung (%)
Projektskizze & SDP	30	22	-8	-27%
Anforderungsanalyse	70	58	-12	-17%
Grobdesign	50	38	-12	-24%
Detailed Design (SDD)	60	45	-15	-25%
Implementierung	150	122	-28	-19%
Testing & Verifikation	95	55	-40	-42%
QA & Dokumentation	75	48	-27	-36%
GESAMT (ohne Puffer)	530	392	-138	-26%
Mit Puffer (15%)	625	449	-176	-28%

Fazit

Die tatsächliche Projektdauer war deutlich geringer als geschätzt. Der Aufwand fiel um 29% unter die Planung. Dies ist auf mehrere Faktoren zurückzuführen: optimierte Prozesse, weniger komplexe technische Anforderungen als erwartet und fokussierte Implementierung durch das Core-Team.

2. Tatsächliche Aufwandsverteilung nach Team

Rolle	Name	Geschätzt (h)	Tatsächlich (h)	Δ (h)	Δ (%)	Status
Scrum Master	Michael Steer	120	140	+20	+17%	Über Plan (Entwicklung)
Product Owner	Luke Engehardt	120	92	-28	-23%	Unter Plan
Developer (Lead)	Giuliana Carrano	120	130	+10	+8%	Leicht über Plan
Developer	Paul Strasser	110	15	-95	-86%	Sehr gering
Developer	Roman Faber	110	15	-95	-86%	Sehr gering
GESAMT		580	392	-188	-32%	Unter Plan

Analyse der einzelnen Aufwände

Michael Steer (140h vs. 120h = +17%): - War nicht nur Scrum Master, sondern auch in der Entwicklung involviert - Unterstützte Giuliana Carrano und übernahm Frontend-Development-Tasks - Koordination, technisches Mentoring und aktive Entwicklungsbeteiligung - Zusätzliche 20 Stunden durch aktive Code-Arbeit gerechtfertigt - Status: Über Plan, aber als Mitentwickler notwendig

Luke Engehardt (92h vs. 120h = -23%): - Requirements weniger kontrovers als erwartet - Stakeholder-Feedback schneller integrierbar - Weniger Change-Request-Management nötig - Status: Gut unter Plan

Giuliana Carrano (130h vs. 120h = +8%): - Lead Developer für Frontend-Development und Datenmodell-Design - Frontend-Entwicklung war effizienter als initial befürchtet - Mit Michael Steers Unterstützung bei Entwicklung konnte Workload besser verteilt werden - Trotz Paul Strasser und Roman Faber Underallocation nur leicht über Plan - Status: Leicht über Plan, Aufwand proportional zur tatsächlichen Komplexität

Paul Strasser (15h vs. 110h = -86% MASSIVE UNDERALLOCATION): - Nur 15 Stunden tatsächlich geleistet (ca. 14% der geplanten Zeit) - Geplant: Datenhaltung & Datenmodell-Design (110h) - Tatsächlich: Minimale Beiträge, hauptsächlich von Giuliana Carrano übernommen - Ursache: Andere Verpflichtungen, nicht verfügbare Kapazität im Projekt - Status: Schwere Abweichung - Planungs-Fehler bei Ressourcenallokation

Roman Faber (15h vs. 110h = -86% MASSIVE UNDERALLOCATION): - Nur 15 Stunden tatsächlich geleistet (ca. 14% der geplanten Zeit) - Geplant: Integration & Testing-Koordination (110h)

- Tatsächlich: Nur minimale QA-Dokumentation und Spot-Checks - Test-Framework wurde vorgefertigt/extern genutzt - Ursache: Ähnlich wie Paul Strasser, mangelnde Verfügbarkeit - Status: Schwere Abweichung - Planungs-Fehler bei Ressourcenallokation

3. Detaillierte Aufwandsanalyse nach Use Cases

UC	Beschreibung	Geschätzt	Tatsächlich	Δ	Status	Bemerkung
UC01	Registrierung	19h	14h	-5h	Unter	Weniger Validierung als erwartet
UC02	Passwort zurücksetzen	13h	0h	-13h	NICHT IMPL	Nicht prioritiert, deferred
UC03	Profil & Einstellungen verwalten	16h	12h	-4h	Unter	Vereinfachte Session-Management
UC04	Quest starten	28h	20h	-8h	Unter	Einfachere Logik als geplant
UC05	Quest abschließen	24h	18h	-6h	Unter	Standardisierte Completion-Logik
UC06	Timer	20h	8h	-12h	Unter	Timer-Pause nicht implementiert
UC07	Noten verwalten	22h	16h	-6h	Unter	Vereinfachte Datenverwaltung
UC08	Dashboard	27h	25h	-2h	Unter	Wie geplant
UC09	Benachrichtigungen	15h	6h	-9h	Unter	Basis-Implementierung, keine Erweiterungen
UC10	CSV Import/Export	19h	18h	-1h	Unter	Einfaches Parsing reichte
UC11	Achievements	15h	10h	-5h	Unter	Fewer edge cases
UC12	Leaderboard	23h	18h	-5h	Unter	Lokale Sortierung statt vollständig dynamisch
UC13	Quest-Katalog verwalten	13h	10h	-3h	Unter	Bestandteile von UC03 wiedergenutzt
UC14	XP- & Levelregeln konfigurieren	27h	19h	-8h	Unter	Admin-Interface vereinfacht

UC	Beschreibung	Geschätzt	Tatsächlich	Δ	Status	Bemerkung
UC15	Benutzerkonten administrieren	19h	12h	-7h	Unter	Config-Datei-basiert statt UI-gesteuert
	SUMME	300h	206h	-94h	-31%	Deutlich unter Budget

Erkenntnisse

- UC02 (Passwort-Reset) wurde nicht implementiert - klare Deviation
- Implementierung insgesamt 32% effizienter als geplant
- Viele Features einfacher zu bauen als initial geschätzt
- Team konnte Code-Wiederverwendung besser nutzen

4. Aufwand nach Aktivitätstypen

Aktivität	Geschätzt (h)	Tatsächlich (h)	Δ (%)	Grund der Abweichung
Code-Implementierung	157	98	-38%	Einfachere Architektur, Code-Templates
Testing & Verifikation	80	35	-56%	Manuelle Tests ausreichend, keine Bugs
Anforderungsanalyse	70	48	-31%	Klare Anforderungen, weniger Iterations
Design & Architektur	110	70	-36%	Standard-3-Schichten-Modell, keine Anpassungen
Dokumentation	100	75	-25%	Weniger Prozess-Dokumentation nötig
Code Reviews & Meetings	50	25	-50%	Standardisierte Code-Größe, weniger Diskussionen

Beobachtung: Die größten Einsparungen (56%, 50%) bei technischem Testing und Reviews. Dies deutet auf gute Initial-Planung und weniger Nacharbeiten hin.

5. Meilenstein-Erfüllung

Meilenstein	Geplant	Tatsächlich	Δ (Tage)	Status	Bemerkung
M1 – Projektskizze	30.10.2025	28.10.2025	-2	Früh	Kick-off schneller

Meilenstein	Geplant	Tatsächlich	Δ (Tage)	Status	Bemerkung
M2 – SDP	06.11.2025	05.11.2025	-1	Früh	Keine Fragen offen
M3 – Requirements	04.12.2025	01.12.2025	-3	Früh	Anforderungen schneller gelöst
M4 – Grobdesign	18.12.2025	10.12.2025	-8	Früh	UML schneller finalisiert
M5 – Prototyp	23.01.2026	18.01.2026	-5	Früh	Implementation ahead of schedule
M6 – SDD & Test-Plan	15.02.2026	14.02.2026	-1	Pünktlich	wie geplant
M7 – Finalisierung	10.03.2026	08.03.2026	-2	Früh	Alle Aufgaben erledigt

6. Risiken: Geplant vs. Realisiert

Risiko	Eintritts-WK (geplant)	Realisiert?	Tatsächliche Auswirkung	Puffer (h)
Unklare Requirements	30%	Nein	0h	20h
Technische Probleme	40%	Ja (minimal)	+2h Debugging	5h
Teamausfälle	25%	Nein (aber Underperformance P.S., R.F.)	-35h U-Pers., -33h U-Faber	10h
Scope Creep	15%	Nein	0h	5h
SUMME RISIKOAUSWIRKUNG			-66h	40h / 75h genutzt (53%)

Der größte "Risiko"-Faktor war nicht ein klassisches Risiko, sondern die deutlich geringere Beteiligung von Paul Strasser und Roman Faber. Dies führte zu unerwarteter Effizienzsteigerung bei den aktiven Entwicklern.

7. Abweichungsanalyse & Hauptgründe – Diskussion Initial vs. Final

Warum war der Aufwand 23% unter Planung? – Detaillierte Analyse

1. Optimierte Implementierung (–38% der geplanten Impl.-Zeit)

Geschätzt: 157h | **Tatsächlich:** 98h | **Abweichung:** –59h

Gründe für die Reduktion: - **Code-Wiederverwendung:** Während der Planung wurde die Komplexität von UI-Komponenten überschätzt. Tatsächlich konnten viele Dashboard-, Leaderboard- und Admin-Components aus standardisierten CSS-Templates wiederverwendet werden. - **Einfachere Datenstrukturen:** Initialschätzung angenommen komplexere Race-Condition-Handling im Timer (UC06). Realität: LocalStorage-basiertes Locking reichte aus. - **Weniger Error-Handling:** Fehlerannahmen zu pessimistisch. Nutzerszenarios (Schulkontext) erwiesen sich als vorhersehbarer als erwartet. - **Keine Backend-Integration nötig:** Initialszenario: Evtl. REST-API-Integration im SDP as Option. Realität: Pure Client-Side reichte völlig aus.

Lerneffekt: Für ähnliche Web-Apps: Implementierungs-Faktor um 25% reduzieren wenn keine Backend-Integration und stabile APIs.

2. Reduziertes Testing (–56% der geplanten Test-Zeit)

Geschätzt: 80h | **Tatsächlich:** 35h | **Abweichung:** –45h

Gründe für die Reduktion: - **Frühe Qualitätskultur:** Von Anfang an Test-Driven innerhalb der Entwicklung. Bugs wurden sofort behoben, nicht am Ende gehäuft gefunden. - **Codebase-Größe:** Initiale Schätzung basierte auf 15 Full-Service UCs. Tatsächliche UC02 Deferred → 14 UCs implementiert → kleinere Testoberfläche. - **Keine Test-Automatisierung geplant, Manuelle Tests reichten:** Test-Framework wurde vorbereitet, aber manuell mit Checklisten lief effizienter für kleine Features. - **Lückenlos dokumentierte Test-Cases:** Beim Schreiben der Test-Dokumentation wurden viele Edge-Cases sofort als "\"non-critical\"" aus dem UC-Acceptance-Scope entfernt.

Lerneffekt: Testing war initial zu konservativ geschätzt. Für ähnliche Projekte: 5-8 Stunden Testing pro UC statt 15-18 Stunden.

3. Schlanke Dokumentation (–25% der geplanten Doc.-Zeit)

Geschätzt: 60h | **Tatsächlich:** 48h | **Abweichung:** –12h

Gründe für die Reduktion: - **Automatisierte Dokumentation:** Viele Konfigdateien und README-Dateien waren boilerplate und wurden durch Scripts generiert. - **Code-ähnliche Struktur:** Lage der Feature-Files war offensichtlich (css/auth.css, js/auth.js) → weniger Dokumentation nötig. - **Fokus auf UC-Dokumentation statt Prozess-Doku:** Prozess-Doku (Team-Charter, Rollen) wurde als Standardtemplate behandelt.

Lerneffekt: Realistische Dokumentation: 40-45h für ähnliche Projekte ausreichend.

4. Effiziente Anforderungsklä rung (–31% der geplanten Analysis-Zeit)

Geschätzt: 70h | **Tatsächlich:** 48h | **Abweichung:** –22h

Gründe für die Reduktion: - **Frühe Konkretisierung der Use Cases:** Anforderungen (Requirements.md) waren gut strukturiert. Keine Mehrdeutigkeiten, die zu Nachfragen

fürten. - **Keine Scope-Changes während Implementierung:** Bestätigt, dass initiales Scoping hoch-qualität war. - **UC02 proaktiv deferred:** Anforderungskklärung zu UC02 (Passwort-Reset) erfolgte sehr früh mit Entscheidung \"nicht in v1.0 implementiert\". - **Stakeholder-Feedback einmal pro Sprint:** Anstatt ad-hoc Fragen wurden Feedback-Slots eingeplant, reduzierte Unterbrechungen.

Lerneffekt: Requirements-Aufwand gut geschätzt, aber bei klaren Anforderungen 30-35% Reduktion realistisch.

5. Hohe Team-Produktivität der aktiven Entwickler (–20% Overhead)

Geplant für 5 Entwickler: 495h | **Tatsächlich 3 Hauptentwickler:** 268h

Gründe: - **Michael Steer & Luke Engelhardt:** Effektive Koordination, weniger Meetings-als-geplant (Daily Standup dauerte 8-10 Min statt geplant 15-20 Min). - **Giuliana Carrano:** Extrem productive Flow-State, kein Context-Switching. Lead Dev mit klarer Ownership → schnelle Entscheidungen. - **Pair-Programming minimal nötig:** Code wurde einfach genug, dass Reviews schneller als geplant liefen.

Impact: Aktivitäten für 3 Personen (Steer, Engelhardt, Carrano) dauerten nicht 3x sondern ~2,8x länger → 11% Effizienzgewinnung.

8. Funktion Points – Retrospektive Neubeurteilung

Initial geschätzte Function Points: 15 UC, ca. 50-60 FP (LOW bis MEDIUM)

Realisierte Komplexität: Durchschnittlich MEDIUM FP

Faktor: 1 FP ≈ 8-10 Stunden (Annahme)

Kalibrierung danach: 1 FP ≈ 6-7 Stunden (tatsächlich)

Empfehlung für nächste Projekte: Faktor-Anpassung im SDP dokumentieren

9. Lessons Learned & Verbesserungen

Was war gut

- Schätzkonferenzen waren effektiv
- Risik-Reserve war dimensioniert (53% genutzt)
- Meilenstein-Tracking funktionierte gut
- Team-Kommunikation war transparent

Was war nicht gut

- Paul Strasser & Roman Faber unzureichend eingespannt
- Passwort-Reset wurde ignoriert (Deviation)

- Testing stark unterschätzt (zu konservativ initial)

Für zukünftige Projekte

- Individuelle Kapazitätsplanung pro Person
 - Strikte Scoping vor Anforderungsanalyse
 - Risk-Review nach M3 und M5
 - Testing-Automatisierung evaluieren (würde Kosten sparen)
-